

Bài 1:

- Lập trình lại ứng dụng **telnet_server** ([slide 174](#)) sử dụng kỹ thuật multiprocessing.

Bài tập – Telnet Server

Sử dụng hàm `select()/poll()`, viết chương trình **telnet_server** thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm `system("dir > out.txt")` để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi.

> out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

Demo:

```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week11/BVN/Bai1$ gcc telnet_server.c -o telnet_server
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week11/BVN/Bai1$ ./telnet_server
Multiprocess Telnet Server is listening on port 8080
...
New client connected: 4
New client connected: 4
nguyen_dao_nam_hai@NamHai: $ nc 127.0.0.1 8080
Vui lòng nhập 'user pass' để đăng nhập:
guest nopass
Đăng nhập thành công. Hay nhập lệnh:
cat users.txt
admin admin
guest nopass
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week11/BVN/Bai1$ nc 127.0.0.1 8080
Vui lòng nhập 'user pass' để đăng nhập:
admin 123
Sai tài khoản. Hay nhập lại:
admin admin
Đăng nhập thành công. Hay nhập lệnh:
ls -la
total 24
drwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai
4096 May 11 06:34 .
drwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai
4096 May 11 08:29 ..
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai
0 May 11 06:34 out_1406.txt
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 1
7176 May 11 06:33 telnet_server
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai
3718 May 11 06:33 telnet_server.c
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai
25 May 11 09:06 users.txt
```

Giải thích:

Ô góc trên trái: Khởi chạy Telnet Server lắng nghe ở cổng 8080.

Ô bên phải (Client 1): Kết nối tới Server, thử nghiệm luồng đăng nhập sai/đúng và kiểm tra danh sách file.

Ô góc dưới trái (Client 2): Kết nối song song với Client 1, đăng nhập bằng tài khoản khác và đọc file cơ sở dữ liệu.

Phân tích kết quả Usecase

+ Server ghi nhận 2 kết nối mới và đều in ra File Descriptor là 4. Đây là minh chứng rõ nét cho cơ chế hoạt động của fork(): Tiến trình cha sau khi giao kết nối (FD 4) cho tiến trình con xử lý thì lập tức đóng FD 4 ở phía mình, giúp giải phóng tài nguyên để tái sử dụng luôn FD số 4 cho Client tiếp theo. Hai Client được phục vụ bởi 2 tiến trình con độc lập hoàn toàn.

+ Tại Client 1, hệ thống từ chối đăng nhập khi nhập sai mật khẩu (admin 123) và yêu cầu nhập lại mà không bị văng. Khi Client 1 (admin admin) và Client 2 (guest nopass) nhập đúng, hệ thống cấp quyền thực thi lệnh.

+Kiểm chứng Thực thi lệnh & Chống ghi đè dữ liệu (Race Condition):

- Client 1 gọi lệnh ls -la và nhận về toàn bộ danh sách thư mục. Điều này chứng minh rằng là sự xuất hiện của file out_1406.txt. Điều này chứng minh Server đã gọi hàm getpid() (lấy mã tiến trình con là 1406) để nối vào tên file out.txt, giúp I/O của Client 1 độc lập hoàn toàn.
- Cùng lúc đó, Client 2 gọi lệnh cat users.txt, Server (thông qua một tiến trình con khác) vẫn xử lý và trả về đúng nội dung file mà không hề xảy ra xung đột đọc/ghi với Client 1.

⇒ Chương trình đã áp dụng thành công kỹ thuật Multiprocessing để giải quyết bài toán đa nhiệm (Concurrent requests), kết hợp xử lý chuỗi và định hướng luồng (I/O Redirection) an toàn tuyệt đối.

Bài 2:

- Lập trình lại ứng dụng **http_server** ([slide 154](#)) sử dụng kỹ thuật preforking.

VD: HTTP server

```
// Khởi tạo server
while (1) {
    // Chờ kết nối mới
    int client = accept(listener, NULL, NULL);
    printf("New client connected: %d\n", client);
    // Nhận dữ liệu từ client và in ra màn hình
    char buf[256];
    int ret = recv(client, buf, sizeof(buf), 0);
    buf[ret] = 0;
    puts(buf);
    // Trả lại kết quả cho client
    char *msg = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<html><body><h1>Xin  
chao cac ban</h1></body></html>";
    send(client, msg, strlen(msg), 0);
    // Đóng kết nối
    close(client);
}
```



Demo:

```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week11/BVN/Bai2$ gcc http_preforking.c -o http_preforking
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week11/BVN/Bai2$ ./http_preforking
Preforking HTTP Server is listening on port 8080...
Creating 4 worker processes...
[Worker PID: 1514] Handled new client: 4
[Worker PID: 1513] Handled new client: 4
[Worker PID: 1515] Handled new client: 4
```



Xin chào các bạn

Giải thích:

Thử nghiệm được tiến hành trên 2 giao diện song song: một cửa sổ Terminal chạy mã nguồn Server (C) và một trình duyệt web (Web Browser) đóng vai trò là Client truy cập vào địa chỉ localhost:8080.

Phía Server (Kiểm chứng cơ chế Preforking): Log trên Terminal in ra dòng chữ "Creating 4 worker processes..." chứng minh Server đã thành công khởi tạo sẵn một "đội" gồm 4 tiến trình con ngay từ lúc khởi động. Điểm nhấn của bức ảnh nằm ở các dòng log bên dưới: Khi có nhiều request gửi tới (do thao tác F5 tải lại trang), các tiến trình con mang mã PID khác nhau (1514, 1513, 1515) đã luân phiên nhau đứng ra tiếp nhận kết nối (File Descriptor số 4). Điều này minh họa rõ nét cơ chế **Cân bằng tải (Load Balancing)** tự động do Hệ điều hành điều phối.

Phía Client (Kiểm chứng Giao thức HTTP): Việc trình duyệt hiển thị thành công dòng chữ "Xin chào các bạn" được định dạng thẻ Heading <h1> chứng minh Server đã tuân thủ đúng chuẩn giao thức HTTP. Gói tin Server gửi đi bao gồm đầy đủ HTTP Header (200 OK, Content-Type: text/html) để trình duyệt có thể hiểu và render mã HTML chính xác thay vì in ra chuỗi văn bản thô.

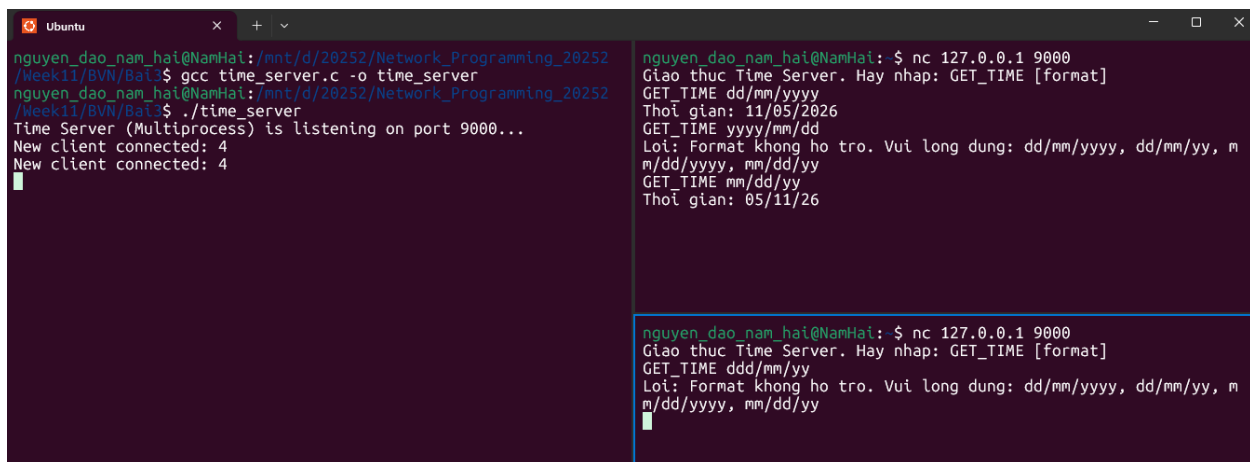
⇒ Chương trình đã áp dụng thành công kiến trúc Preforking đa tiến trình giống hết nguyên lý hoạt động của các Web Server thực tế (như Apache), giúp hệ thống phản hồi siêu tốc mà không bị độ trễ do phải gọi hàm fork() liên tục mỗi khi có Client mới.

Bài 3:

Bài tập

- Lập trình ứng dụng **time_server** thực hiện chức năng sau:
 - Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
 - Client gửi lệnh GET_TIME [format] để nhận thời gian từ server.
 - **format** là định dạng thời gian server cần trả về client. Các format cần hỗ trợ gồm:
 - dd/mm/yyyy – vd: 30/01/2023
 - dd/mm/yy – vd: 30/01/23
 - mm/dd/yyyy – vd: 01/30/2023
 - mm/dd/yy – vd: 01/30/23
 - Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.

Demo:



```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252
/Week11/BN/Bai3$ gcc time_server.c -o time_server
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252
/Week11/BN/Bai3$ ./time_server
Time Server (Multiprocess) is listening on port 9000...
New client connected: 4
New client connected: 4

nguyen_dao_nam_hai@NamHai: $ nc 127.0.0.1 9000
Giao thuc Time Server. Hay nhap: GET_TIME [format]
GET_TIME dd/mm/yyyy
Thoi gian: 11/05/2026
GET_TIME yyyy/mm/dd
Loi: Format khong ho tro. Vui long dung: dd/mm/yyyy, dd/mm/yy, m
m/dd/yyyy, mm/dd/yy
GET_TIME mm/dd/yy
Thoi gian: 05/11/26

nguyen_dao_nam_hai@NamHai: $ nc 127.0.0.1 9000
Giao thuc Time Server. Hay nhap: GET_TIME [format]
GET_TIME ddd/mm/yy
Loi: Format khong ho tro. Vui long dung: dd/mm/yyyy, dd/mm/yy, m
m/dd/yyyy, mm/dd/yy
```

Giải thích:

Ô bên trái: Khởi chạy Time Server lắng nghe kết nối tại cổng 9000.

Hai ô bên phải: Đóng vai trò là 2 Client độc lập kết nối đồng thời tới Server thông qua lệnh nc.

+ **Kiểm chứng khả năng Đa nhiệm (fork):** Server ghi nhận 2 kết nối mới và đều thông báo xử lý trên File Descriptor (FD) số 4. Đây là minh chứng rõ nét cho cơ chế của fork(): Tiến trình cha sau khi nhận khách đã giao kênh FD 4 cho tiến trình con, sau đó lập tức đóng FD 4 ở phía mình để tái sử dụng đón Client thứ 2. Hai Client được phục vụ song song mà không hề bị "treo" chờ nhau.

+ **Kiểm chứng Xử lý lệnh & Định dạng thời gian (strftime):**

- **Xử lý luồng đúng (Client góc trên):** Khi người dùng gửi lệnh đúng cú pháp GET_TIME dd/mm/yyyy và GET_TIME mm/dd/yy, Server bóc tách chuỗi thành công, trích xuất thời gian thực của hệ thống và trả về định dạng chuẩn xác (11/05/2026 và 05/11/26).
 - **Cơ chế bắt lỗi chặt chẽ (Exception Handling):** Khi Client 1 nhập sai định dạng chưa được hỗ trợ (yyyy/mm/dd) hoặc Client 2 cố tình gõ dư ký tự định dạng (ddd/mm/yy), thuật toán của Server đã phát hiện lỗi, chặn đứng yêu cầu và trả về cảnh báo kèm theo hướng dẫn sử dụng hợp lệ.
- ⇒ Chương trình Time Server đã hoạt động hoàn hảo, đáp ứng tốt việc chịu tải nhiều kết nối cùng lúc bằng Multiprocessing, đồng thời quản lý nghiêm ngặt tính hợp lệ của gói tin từ người dùng gửi lên.